JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class Objects

java.lang.Object
    java.util.Objects

```
public final class Objects
extends Object
```

This class consists of `static` utility methods for operating on objects. These utilities include `null`-safe or `null`-tolerant methods for computing the hash code of an object, returning a string for an object, and comparing two objects.

Since:

1.7

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static <T> int	compare(T a, T b, Comparator<? super T> c)	Returns 0 if the arguments are identical and c.compare(a, b) otherwise.
static boolean	deepEquals(Object a, Object b)	Returns true if the arguments are deeply equal to each other and false otherwise.
static boolean	equals(Object a, Object b)	

		Returns `true` if the arguments are equal to each other and `false` otherwise.
`static int`	**`hash`**`(`**`Object`**`... values)`	Generates a hash code for a sequence of input values.
`static int`	**`hashCode`**`(`**`Object`** `o)`	Returns the hash code of a non-`null` argument and 0 for a `null` argument.
`static boolean`	**`isNull`**`(`**`Object`** `obj)`	Returns `true` if the provided reference is `null` otherwise returns `false`.
`static boolean`	**`nonNull`**`(`**`Object`** `obj)`	Returns `true` if the provided reference is non-`null` otherwise returns `false`.
`static <T> T`	**`requireNonNull`**`(T obj)`	Checks that the specified object reference is not `null`.
`static <T> T`	**`requireNonNull`**`(T obj,` **`String`** `message)`	Checks that the specified object reference is not `null` and throws a customized **`NullPointerException`** if it is.
`static <T> T`	**`requireNonNull`**`(T obj,` **`Supplier`**`<`**`String`**`> messageSupplier)`	Checks that the specified object reference is not `null` and throws a customized **`NullPointerException`** if it is.
`static` **`String`**	**`toString`**`(`**`Object`** `o)`	Returns the result of calling `toString` for a non-`null` argument and `"null"` for a `null` argument.

static String	toString(Object o, String nullDefault)	Returns the result of calling toString on the first argument if the first argument is not null and returns the second argument otherwise.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

equals

```
public static boolean equals(Object a,
                             Object b)
```

Returns true if the arguments are equal to each other and false otherwise. Consequently, if both arguments are null, true is returned and if exactly one argument is null, false is returned. Otherwise, equality is determined by using the equals method of the first argument.

Parameters:

a - an object

b - an object to be compared with a for equality

Returns:

true if the arguments are equal to each other and false otherwise

See Also:

Object.equals(Object)

deepEquals

```
public static boolean deepEquals(Object a,
                                 Object b)
```

Returns true if the arguments are deeply equal to each other and false otherwise. Two null values are deeply equal. If both arguments are arrays, the algorithm in Arrays.deepEquals is used to determine equality. Otherwise, equality is determined by using the equals method of the first argument.

Parameters:

`a - an object`

`b - an object to be compared with a for deep equality`

Returns:

`true if the arguments are deeply equal to each other and false otherwise`

See Also:

`Arrays.deepEquals(Object[], Object[])`, `equals(Object, Object)`

hashCode

```
public static int hashCode(Object o)
```

Returns the hash code of a non-`null` argument and 0 for a `null` argument.

Parameters:

`o - an object`

Returns:

`the hash code of a non-null argument and 0 for a null argument`

See Also:

`Object.hashCode()`

hash

```
public static int hash(Object... values)
```

Generates a hash code for a sequence of input values. The hash code is generated as if all the input values were placed into an array, and that array were hashed by calling `Arrays.hashCode(Object[])`.

This method is useful for implementing `Object.hashCode()` on objects containing multiple fields. For example, if an object that has three fields, `x`, `y`, and `z`, one could write:

```
@Override public int hashCode() {
    return Objects.hash(x, y, z);
}
```

Warning: When a single object reference is supplied, the returned value does not equal the hash code of that object reference. This value can be computed by calling `hashCode(Object)`.

Parameters:

`values - the values to be hashed`

Returns:

a hash value of the sequence of input values

See Also:

`Arrays.hashCode(Object[])`, `List.hashCode()`

toString

```
public static String toString(Object o)
```

Returns the result of calling `toString` for a non-`null` argument and `"null"` for a `null` argument.

Parameters:

`o` - an object

Returns:

the result of calling toString for a non-null argument and "null" for a null argument

See Also:

`Object.toString()`, `String.valueOf(Object)`

toString

```
public static String toString(Object o,
                              String nullDefault)
```

Returns the result of calling `toString` on the first argument if the first argument is not `null` and returns the second argument otherwise.

Parameters:

`o` - an object

`nullDefault` - string to return if the first argument is null

Returns:

the result of calling toString on the first argument if it is not null and the second argument otherwise.

See Also:

`toString(Object)`

compare

```
public static <T> int compare(T a,
                              T b,
                              Comparator<? super T> c)
```

Returns 0 if the arguments are identical and `c.compare(a, b)` otherwise. Consequently, if both arguments are `null` 0 is returned.

Note that if one of the arguments is `null`, a `NullPointerException` may or may not be thrown depending on what ordering policy, if any, the `Comparator` chooses to have for `null` values.

Type Parameters:

`T - the type of the objects being compared`

Parameters:

`a - an object`

`b - an object to be compared with a`

`c - the Comparator to compare the first two arguments`

Returns:

`0 if the arguments are identical and c.compare(a, b) otherwise.`

See Also:

`Comparable, Comparator`

requireNonNull

```
public static <T> T requireNonNull(T obj)
```

Checks that the specified object reference is not `null`. This method is designed primarily for doing parameter validation in methods and constructors, as demonstrated below:

```
public Foo(Bar bar) {
    this.bar = Objects.requireNonNull(bar);
}
```

Type Parameters:

`T - the type of the reference`

Parameters:

`obj - the object reference to check for nullity`

Returns:

`obj if not null`

Throws:

NullPointerException - if obj is null

requireNonNull

```
public static <T> T requireNonNull(T obj,
                                   String message)
```

Checks that the specified object reference is not `null` and throws a customized `NullPointerException` if it is. This method is designed primarily for doing parameter validation in methods and constructors with multiple parameters, as demonstrated below:

```
public Foo(Bar bar, Baz baz) {
    this.bar = Objects.requireNonNull(bar, "bar must not be null");
    this.baz = Objects.requireNonNull(baz, "baz must not be null");
}
```

Type Parameters:

T - the type of the reference

Parameters:

obj - the object reference to check for nullity

message - detail message to be used in the event that a NullPointerException is thrown

Returns:

obj if not null

Throws:

NullPointerException - if obj is null

isNull

```
public static boolean isNull(Object obj)
```

Returns `true` if the provided reference is `null` otherwise returns `false`.

API Note:

This method exists to be used as a `Predicate`, `filter(Objects::isNull)`

Parameters:

obj - a reference to be checked against null

Returns:

true if the provided reference is null otherwise false

Since:

1.8

See Also:

Predicate

nonNull

```
public static boolean nonNull(Object obj)
```

Returns `true` if the provided reference is non-`null` otherwise returns `false`.

API Note:

This method exists to be used as a Predicate, `filter(Objects::nonNull)`

Parameters:

`obj` - a reference to be checked against `null`

Returns:

`true` if the provided reference is non-null otherwise `false`

Since:

1.8

See Also:

Predicate

requireNonNull

```
public static <T> T requireNonNull(T obj,
                                   Supplier<String> messageSupplier)
```

Checks that the specified object reference is not `null` and throws a customized `NullPointerException` if it is.

Unlike the method `requireNonNull(Object, String)`, this method allows creation of the message to be deferred until after the null check is made. While this may confer a performance advantage in the non-null case, when deciding to call this method care should be taken that the costs of creating the message supplier are less than the cost of just creating the string message directly.

Type Parameters:

`T` - the type of the reference

Parameters:

`obj` - the object reference to check for nullity

`messageSupplier` - supplier of the detail message to be used in the event that a `NullPointerException` is thrown

Returns:

`obj if not null`

Throws:

`NullPointerException - if obj is null`

Since:

`1.8`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.